	Date	Experiment	Signature
1.	20.07.2026	Basic Networking Command used in Linux & Commands	23/07/26 (9)
2.	28/7/2026	Different types of Network cable	4/8/26 (9)
3.	4/8/26	Cisco Packet Tracer	10/8/26 (10)
4	10/8/26	Setting up a Local Area Network.	(10) 10/8/26
5.	18/8/26	Experiments on packet capture tools : wireshak.	29/9/26 (10)
6	7/9/26	Error detection and error correction	29/9/26 (9)
7.	22/9/26	NMAP to discover Live Hosts using Nmap Scan on the Trythackme platform.	29/9/26 (10)
8	28/9/26	END to END Communictation at Transport layer.	29/9/26 (9)
9	28/9/26	Sliding window protocol	29/9/26 (9)
10.	6/10/26	Implement your own ping program	6/10/26 (9)
11	6/10/26	write a code using RAW sockets to implement packet sniffing	6/10/26 (9)
12.	7/10/26	Implementation of subnetting in Cisco Packet Tracer Simulator.	13/10/26 (9)
13.	7/10/26	Routing of network layer.	13/10/26 (10)
14.	13/10/26	Internetworking with router in cisco packet tracer simulator.	13/10/26 (10)
15.	13/10/26	Analyze various types of servers	13/10/26 (9)

Exp no: 01
Date : 20/7/2026

Basic Networking Command

1. arp -a

Show IP to MAC address mappings [arp -a]

O/P: gateway (10.0.2.2) at 52:54:00:12:35:02 [ether] on enp0s3

2. hostname

display the name of our computer

O/P: localhost.localdomain

3. ipconfig/all

Displays detailed network configuration

O/P: windows IP configuration

Host name ..... : Gayathri
Primary DNS suffix ... :
Node Type .... : Mixed

4. nbstat -a

Trailbleshoot NetBios name resoultion

O/P: Node Address : [192.168.1.100]

Name	Type	Status
Server01	<00> Unique	Registered
Server01	<20> Unique	Registered

5. net stat:

Show network connection, ports, ratis

O/P:

Proto	Recv-Q	Send-Q	Local Address	State
tcp	0	0	192.168.1.100.22	Established

6. netstat -r:

Display the routing table of the computer.

O/P:

21..00 ff eb da 62 66 .. Tap - windows Adapter V9

1 ..... Software loopback Interface 1.

7. nslookup

Performs DNS lookup

O/P:

Default Server: Unknown
Address: 127.0.0.1

8. Pathping

Combines ping and route analysis

O/P:

0	DESKTOP - PC
1	192.168.1.1

9. Ping

Test network Connectivity

O/P: ping [-t] [-a] [-n count] [-l size] [-f] [-i TTL]

10. Route

Displays/manipulates routing table

O/P: [-f] [-p] [-4 -6] command [destination]

Linux Networking Commands

1. ip

Display IP address
Configure network interface
Add / delete IP address
Managing routing tables

2. ip address add

Assigns a new IP address to a network interfaces

Ex: ip address add 192.168.1.254/24

O/P: lo: <LOOPBACK UP, LOWER_UP>

3. Delete an IP Address

Remove the IP address from the interface

Eg: ip address del 192.168.1.254/24 dev enp0s3

O/P:

enp0s3 : <BROADCAST, MULTICAST, UP, LOWER_UP>
inet 10.0.2.15/24 brd 10.0.2.255 scope
global dynamic enp0s3

4. Bring Interface Online

Enables the network interface

Eg: ip link set enp0s3 up

O/P: enp0s3: mtu 1500 qdisc - fq - codel state

5. Interface online

Displays the network interface

Eg: ip link set enp0s3 down

o/p:

enpos3 : <BROADCAST, MULTICAST, DOWN, LOWER_UP>

6. Enable promiscous Mode

Allows the interface recieve all network pack

Eg: ip link set enpos3 promisc on

o/p: enpos3 : <BROADCAST, MULTICAST, PROMISC>

7. Add a default route

Adds the dfault gateway for all network traffic

o/p:

default via 192.168.1.254 dev enpos3

8. Add a route through a gateway

Routes traffic traffic for the 192.168.1.0/24 network through gateway 192.168.1.254

o/p: ip route add 192.168.1.0/24 dev etho

9. Delete a route

Delete the route to the 192.168.1.0/24 network through gateway 192.168.1.0/24

Eg: ip route delete 192.168.1.0/24 via 192.168.1.254

. pf Config

Configuring & Troubleshoot Network.

o/p: Media & state ... : Media disconnected

11. mtr

Combines the functionality of the ping and Traceroute Commands

o/p: Host

1. 172.16.8.1 Loss % 0.0 134 0.1 0.2 0.1

12 a) mtr

b) mtr -g google.com [Show numeric IP address]

c) mtr -b google.com [numeric ip address & hostnames]

d) mtr -c 10 google.com [numeric of ping that you want]

13. tcpdump

Designed for capturing & displaying packets

i) tcpdump -i etho -c 10 host 8.8.8.8

Filter our traffic coming from a specific ~~that~~ host.

14. Ping

Display best way to test connectivity b/t 2 nodes.

o/p:

Minimum = 0ms, Maximum = 0ms, Average = 0ms

Result:

The basic Networking & Linux Commands are learned & executed successfully.

27/01/26

Ex no 02
Date : 28/7/2025

Different type of Network Cables.

Aim:

To study of different types of Network Cables

1) UTP cable
2) STP cable
3) Coaxial cable
4) Fibre optic cable

Cable type	category	Max data transmission	Adv/ Dis	Application
UTP	Category 3	10 bps	Adv: • Cheaper in cost • Easy to install Dis: • More prone to EMI	10Base-T Ethern-
	category 5	up to 100 Mbps		Fast ethernet Gigabit ethern
	Category 5e	1 Gbps		Fast ethernet
STP	Category 6, 6a	10 Gbps	Adv: • Shilded • Faster than UTP • Less susceptible to noise Dis: • Expensive • Greater Installation effort	Gigabit Ethernet 10 G Ethernet (55 m) widely used in data centres
STP	Category 7	10 Gbps		Gigabit Ethernet, 10 G Ethernet (100m)

Coaxial Cable	RG-6 RG-59 RG-11	10-100Mbps	• High bandwidth • Immune to interferance • Low loss bandwidth • versatile Dis: • Limited distance • Cost • size is bulky	Spread of signal is 500m Television network High speed internet connections
Fibre optic Cable	~~Single mode~~ Multimode	100Gbps	Adv: • High speed • High bandwidth • High security • Long distance Dis: • Expensive • Requires skilled installers	• Maximum distance of fibre optics cable is arround 100m

Result:

The different types of network cables are learned and created successfully.

4/8/22

Ex no: 03

Date: 4/8/26

CISCO PACKET TRACER

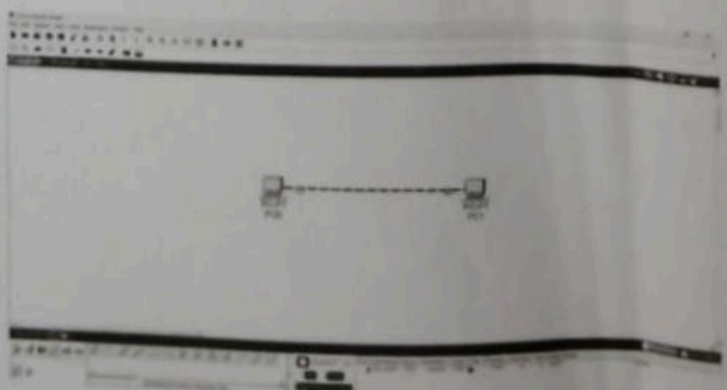
Analyse the behaviour of network device using CISCO PACKET TRACER Emulator.

1. From the network component box, click & drag and drop the below components.

a. 4 Generic pcs and one Hub

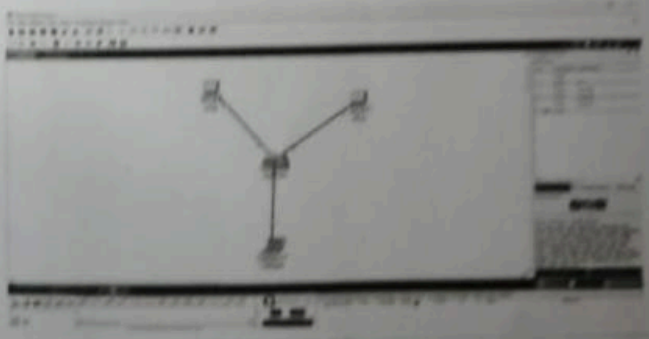
2. Click on connection

* Click on copper Straight - through cable
* Select one of the PC & connect it to HUB using the cable.
* Similarly connect 4pcs to the HUB using copper straight - through cable.



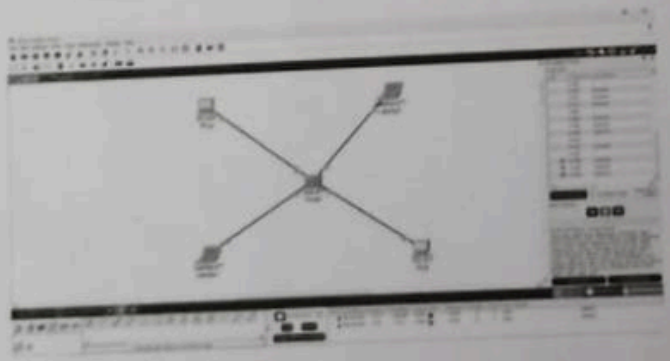
* Generate pcs and one switch

* Click on copper straight through cable
* Select one of the pc and connect it to switch using the cable.
* Similarly connect 2pcs to the switch using copper straight - through cable.
* click on the pcs connected to switch go to the desktop tab, click on IP configuration and enter an IP address
* Set the message box to one pcs to another pc.



C PC to PC

* Connect PC_1 and PC_2 using an Ethernet cable
* Assign IP address to both PCs in the same network.
* Check the connection using ping b/t PC_1 and PC_2
* If the ping succeeds the PC-to-PC connection is established.



Result:

CISCO PACKET TRACKER are learned & executed Successfully.

10/8/20

Exno: 04
Date: 10/8/24 Setting up a Local Area Network:

Aim:

To design, setup and configure a LAN connecting four computers through a network switch, assign static IP address to all devices. verify connectivity using the ping command & demonstrate resource data storage across the network.

Diagram:

LAN was implemented using STAR topology.

Implementation:

Step 1: Place the device

Open the Cisco packet Tracer

From the device category panel, select End device & drag 4 PCs onto the workspace.

Step 2: Cable the device

Click connections → Select copper straight through cable.

Click each PC, then click the switch and select an available Fastethernet port to complete to each connection.

Step 3: Assign IP Address to PCs

Click a PC → Desktop tab → IP configuration

Enter the static IP Address & Submit mask as per the IP table for each PC

Step 4:

Click a Switch → CLI Tab

Commands:

```
Enable
Configure Terminal
Interface vlan1
ip address 10.1.1.5 255.0.0.0
no shutdown
exit
```

Step 5:

verify connectivity using ping

Click a PC → Desktop tab → Command prompt

use the Ping Command to test connectivity to the other PC,

Successful 'Reply from ..." message confirm Connectivity

Step 6: Demonstrate data sharing

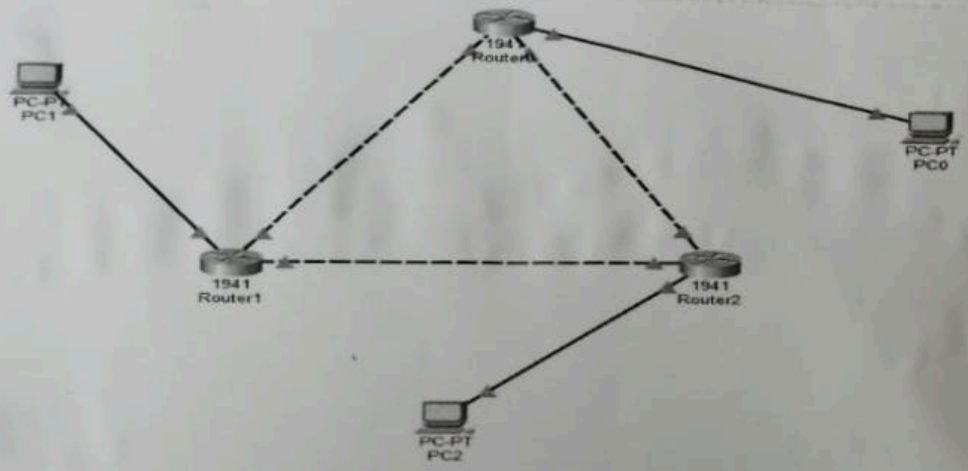
Add a server device via PTS and connect it to the switch.

Step 7: observe packet flow in Simulation work.

Switch to simulation mode

Send a ping or web request b/t two devices

Use the play controls to step through the switch, as it travels hop-by-hop through the switch observing MAC address based generating.



Result:

Setting up a Local Area Network (LAN) we learned & Excecuted Successfully.

10/8/22

Ex:No:05
Date:18/9/24 Experiments on packet capture tools: wireshark

Capturing & Analyzing packets using wireshark tool:

To filter, capture, view, packets in wireshark tool. Capture 100 packets from ethernet: IEE 802.3 LAN Interface & save it.

Procedure:

- Select Local Area Connection in wireshark
- Go to capture option
- Select stop capture automatically after 100 packets.
- Then click start capture
- Save the packet.

1. Create a filter to display only TCP/UDP packe
Inspect the packets & provide the flow graph

Procedure:

- Select LAN connection is wireshark.
- Go to capture options
- Selects stop capture automatically after 100 packets.

Then click the start capture.

Save the packet.

2. Create a filter to display only ARP packets and Inspect the packets.

Procedure:

Go to Capture option

Select stop capture automatically after 100 packets.

Then click start capture.

Search ARP packets in search bar.

Save the packets.

3. Create a file to display only DNS packets & provide the flow graph.

Procedure:

Go to capture option

Select Stop capture automatically after 100 packets.

Then click Start capture.

Search DNS packets in Search bar

To see the flow graph click Statistics flow graph.

Result:

Capturing and Analysing packets using wireshark are studied & executed Successfully.

Ex NO: 06 Error Dection & Error Correction
Date: 7/9/20

Aim:

Write a program to implement error detection & correction using HAMMING CODE Concept. Make a test run to input data stream & error correction features.

Sender:

* Input to sender file should be a txt of any length.

* Apply hamming code concepts on the binary data & add redundant bits to it.

Code:

```
data = input ("Enter a data:")
m = len(data)
r=0
while 2**r <(m+r+1):
    r+=1
hamming = [0]* (m+r+1)
j=0
for i in range (1,m+r+1):
    if i & (i-1) !=0
    hamming[i]= int (data[j])
    j+=1
```

```
for i in range(3):
    pos= 2**i
    parity=0
    for j in range (1, m+r+1):
        if j &pos !=0
            parity ^= hamming[j]
    hamming[pos]= parity
```

Receiver:

Receiver program should read the input from channel file.

Apply hamming code on the binary data to check for error

If there is an error, display the position of the error.

Code.

```
Receiver = input ("Enter the received hamming
                  code ")
n = len(receiver)
hamming =[0]*(n+1)
for i in range (1, n+1)
    hamming[i]= int (receives[n-i])
r=0
while 2**r <n+1:
    r+=1
error=0
```

```
for i in range(r):
    pos = 2**i
    parity = 0
    for j in range(1, n+1):
        if j & pos != 0:
            parity ^= hamming[j]
    error += parity*pos

if error != 0:
    print("Error detected", error)
else:
    print("There is No Error")

for i in range(n, 0, -1):
    print(hamming[i], end="")

print()
```

Sample Input:

Data bits: 1011

Hamming Code: 1100110

Receive Hamming code: 1000110

Error detected at position: 6

Correct Hamming code: 1100110

Result:

Error detection & Error Correction using Hamming Code are learned & executed successfully.

29/9/26

Ex no: 07
Date: 22/9/25

NMAP to Discover Live Hosts Using Nmap Scans on the TryHackme Platform.

Aim:

To discover like hosts using Nmap Scans [ARP, ICMP, TCP/UDP] on the TryHackme platform Room link; https://tryhackme.com/room/nmapol.

Procedure:

Task 1: Introduction

* ARP Scan:
* ICMP Scan
* TCP/UDP

Task 2: Subnetworks

Network Segment - It is a collection of computers linked togther through a common medium.

Subnetworks: It is usually the equivalent of one or more networks together and configured to utilize the same router in IP networks.

* Subnets with /16
* Subnets with /24

Task 3: Enumerating Targets

We need to identify the targets

we want to scan before we conduct any "action"

1) List
- < MACHINE - IP > < Scanme . nmap . org > < or >
- will scan 3 IP addresses

2) range
- 10.11.12.15-20
- will scan 6 IP addresses

3) subnet
- < MACHINE - IP > / 30
- will scan 4 IP addresses

nmap -sL < TARGETS >

Task 4: Discovering like hosts.

1) ARP from link layer
2) ICMP from network layer
3) TCP from transport layer
4) UDP from transport layer.

Task 5: Nmap host Discovery using ARP

1) when a privilidged user tries to scan LAN, Nmap uses ARP requests

2) To discover internet hosts without port scanning like systems use Nmap targets.

Task 6: Nmap host discovery using ICMP

1) Add -PE option to utilize ICMP echo request to discover live hosts.

2) Nmap -PE -sn Target to check host is live.

3) -PP option instructs NMAP to use ICMP timestamp requests.

nmap -PP -sn TARGET.

Task 7: using Reverse -DNS Cookup

1) Nmap is to use reverse -DNS internet hosts.

2) -R option to query the DNS server even for offline hosts.

Task 8: Summary

ARP Scan, ICMP echo, SCAN, ICMP time SCAN, ICMP address mask scan, TCP SYN ping Scan, TCP Ack ping Scan, UDP ping Scan.

Result:

Nmap to discover live host using tryhackme platform was successfully executed.

24/9/24

Ex:08 END to END Communication at Transport layer.

Date 28/9/24

a)

Aim:

Implement echo client server using TCP/UDP targets.

Procedure:

* Create a socket using python socket programming.
* Specify the IP addresses and port number.
* Bind the server listen for client connections
* Make the server listen for client connections
* Connect the client to the server.
* Send & receive messages behaviour client & server
* Display the received message.
* Close the socket Connection.

Code:

TCP Server:

```
import socket
server = socket.socket (socket.AF_INET, socket.SOCK_STREAM)
server.bind ("127.0.0.1", 5000))
```

```
server.listen(1)
print("server is waiting for connection")
conn, addr = server.accept()
print("Client Connected")
while True:
    data = conn.recv(1024).decode()
    if not data:
        break
    print("client:", data)
    conn.send(data.encode())
conn.close()
server.close()
```

TCP_Receiver(client)

```
import socket
client = socket.socket(socket.AF_INET,
socket.SOCK_STREAM)
client.connect(("127.0.0.1", 5000))
message = input("Enter msg")
client.send(message.encode())
reply = client.recv(1024).decode()
print("server:", reply)
client.close()
```

Terminal 1: Server

Python tcp_server.py

Server is waiting for connection.....

Client Connected

Client : Hello.

Terminal 2: client :

python tcp_client.py

Enter message : Hello

Server : Hello

TCP echo is working

Result:

The TCP/UDP packets using echo client Server socket program is executed Successfully

2019/24

EX: 09 SLIDING WINDOW PROTOCOL

Date 28/9/26

Aim

Write a program to implement flow control at data link layer using sliding window protocol. Simulate the flows from one node to another.

Code:

```
import time
window = int(input("Enter window size : "))
message = input("Enter message")
start=0
while (start < len(message)):
print ("SENDER")
end = start window
if end > len (message):
    end = len (message)
# sender buffer.
Sender_buffer = [ ]
for i in range (start, end):
    frame = (i, message[i])
    sender_buffer.append (frame)
    print ("Sender buffer")
# Delay
    Print ("In waiting for receiver :)")
    time.sleep(1)
```

```
# Receiver
    print ("RECEIVER")
    expected = start
    error = false
    for i in sender - buffer:
    frame - no = frame[0]
    data = frame[1]

# Receiver buffer
  if error:
      receiver buffer = ("NACK", expected)
  else
    ack = expected + 1
```

Output:

Enter window size : 3

Enter message : Hello

SENDER

Frame no: 0 Data: H
Frame no: 1 Data: e
Frame no: 2 Data: l
Frame no:

Sender Buffer [(0,'H'), (1,'e'), (2,'l')]

RECEIVER:

R.F : 0 Data: H
R.F : 1 " : e
R.F : 2 " : l

Received Buffer ('ACK', 2)

Result:

Sliding window Protocol is learned & executed successfully

28/9/24

Ex: 10

Date: 6/10/24 Implement your own ping program.

Aim:

To implement a ping program that checks the connectivity between two devices over a network and measures the response time.

Algorithm:

* Start the program
* Get the destination IP address on hostname from the user.
* Send on ICMP Echo request to the destination
* wait for the ICMP echo reply.
* Calculate the display response time.
* Repeat the process for a specified number of requests.
* Stop the requests.

Code:

```
Import os
import platform.
import subprocess
def ping(host = "google.com", count=4):
    param = "-n" if platform.system().lower()
        == "windows" else "-c"
    Command = ["ping" param, str(count),
                host]
    trs:
        output = subprocess.check_output(
                Command, universal_newlines=True)
```

```
        print (output)
    except Expectation as e
        print (f"ping failed: {e}")
if __name__ == "__main__":
    ping("8.8.8.8")
```

Output:

```
Pinging 8.8.8.8 with 32 bytes of data:
Reply from 8.8.8.8 : bytes = 32 time = 70ms, TTL = 119
Reply from 8.8.8.8 bytes = 32 time, time = 107ms TTL = 172
Reply from 8.8.8.8 : bytes = 32 time = 48ms TTL = 119
Reply from 8.8.8.8 bytes = 32 time = 88ms TTL = 119.
Ping Statistics for 8.8.8.8
    Packets : sent = 4, Received = 4, lost = 0 (0% loss)
```

Result:

The ping program was executed successfully and the output got verified.

6/10

Ex: no: 11
Date: 6/10/24

Write a code using RAW sockets to implement packet sniffing.

Aim:

To write a python program using RAW Socket to capture and analyze network packets.

Algorithm:

* Start the program.
* Create a raw socket using SOCKET AF_PACKET and SOCKET.SOCK_RAW.
* capture incoming network packets.
* Extract the Ethernet header from each packet
* Identify the source MAC address, destination MAC address and ethernet protocol.
* Display the packet details.
* Repeat until the program is stopped.

Code:-

```
from scapy.all import sniff
from scapy.layers.inet import IP, TCP, UDP, ICMP
def packet_callback(packet):
    if IP in packet:
        ip_layer = packet[IP]
        protocol = ip_layer.proto
        src_ip = ip_layer.src
        dest_ip = ip_layer.dst
    # Determine the protocol
```

```
protocol_name = = " "
if protocol = = 1
    protocol_name = name = "ICMP"
elif protocol = = 6:
    protocol_name = = "TCP"
elif protocol = = 17:
    protocol_name = "UDP"
print(f" Protocol : {protocol_name}")
print(f" Source IP : {src_ip}")
print(f" Destination IP : {dest_ip}")
sniff(iface = 'wifi', prn = packet_callback, filter =
        "ip", store=0)
```

output :-

Destination IP : 224.0.0.251

Protocol : TCP

Source IP : 172.16.11.15

Destination IP : 172.16.8.1

Protocol : UDP

Source IP : 172.16.11.136

Destination IP : 239.255.255.259

Result:

Thus the python program was successfully executed to check the socket programming messages were successfully sent & received b/t the client & server.

Exp.no: 12
Date: 7/10/24

Implementation of subnetting in CISCO Packet tracer simulator.

Aim:

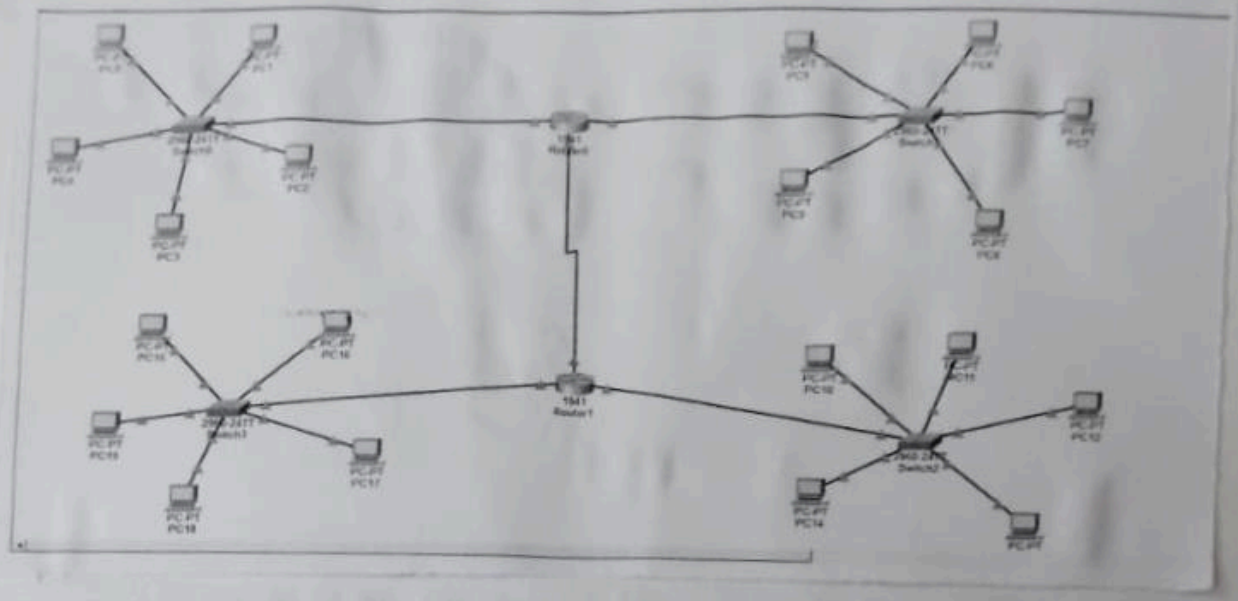
To implement the subnetting in CISCO PACKET TRACER simulator.

Procedure:

* open cisco packet tracer and create a new network topology.
* Add the required routers, switches and PCs to the workspace.
* Connect the device using appropriate copper cable
* use the network 192.168.10.0/24 and divide it into 127 networks.
* Configure the router interface with the required IP address and subnet mask 255.155.255.224.
* use enable and configure terminal in the router CLI.
  i) IP address <IP address> 255.255.255.224
  ii) No shutdown.
* Configure the required switch ports in access mode.
* Assign each PC its IP address subnet mask and default gateway.
* Repeat the configuration for all required Subnets

+ use ping command to test connectivity between PCs and the router.

+ If ping is successfull, the network is configured correctly.



Result:

The network was sucessfully subnetted using the 127 subnet mask in Cisco Packet Tracer. Connectivity between the devices was succesfully verified using Ping command.

Expno: 13
Date: 7/10/26 Routing at Network layer.

a) Simulate static routing protocol configuration using CISCO packet Tracer.

Aim:-

To simulate a static routing configuration using CISCO Packet tracer.

Procedure:

* open Cisco packet Tracer and create the given network topology.

* Connect 3 routers & PCs using suitable cables.

* Enable all router interfaces using no simulation

* Configure static routers on router0, router1, router2 using IP address.

* Set main and backup router using different administrative distance value.

* Test Connectivity

* use tracert to verify the routing path

* Disconnect the main link and verify that the backup route works

* Thus a static routing is sucessfully Configured and verified.

```
Router>enable
Router#show ip route static
S     30.0.0.0/8 [20/0] via 40.0.0.2
S     50.0.0.0/8 [10/0] via 40.0.0.2
S     60.0.0.0/8 [1/0] via 40.0.0.2

Router#
%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0, changed state to down

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0, changed state to up

Router#show ip route static
S     30.0.0.0/8 [10/0] via 20.0.0.2
S     50.0.0.0/8 [10/0] via 40.0.0.2
S     60.0.0.0/8 [1/0] via 40.0.0.2
```

Result:

Thus static routing was sucessfully configured in cisco Packet Tracer. Connectivity between all networks was verified successfully using ping.

b) Simulate RIP using CISCO Packet Tracer.

Aim:

To simulate RIP using CISCO Packet Tracer

Procedure:

* Create the topology with routers & 2PCs
* Connect the device using suitable cables.
* Assign the given IP address to both PCs
* Configure fastethernet and serial interface of all routers
* Enable interface using no shutdown.
* Set clock rate &4000 and bandwidth 64 on DE serial interface.
* Enable RIP on all routers using router rip
* Add directly connected networks using the network command.
* Test connectivity using ping 20.0.0.2
* use tracert 20.0.0.2 to verify the selected route.
* Disconnect the direct router 0 - Router 2 link and use tracert again to verify RIP selects the alternate route.

```
Pinging 20.0.0.2 with 32 bytes of data:

Reply from 20.0.0.2: bytes=32 time=55ms TTL=125
Reply from 20.0.0.2: bytes=32 time=56ms TTL=125
Reply from 20.0.0.2: bytes=32 time=60ms TTL=125
Reply from 20.0.0.2: bytes=32 time=55ms TTL=125

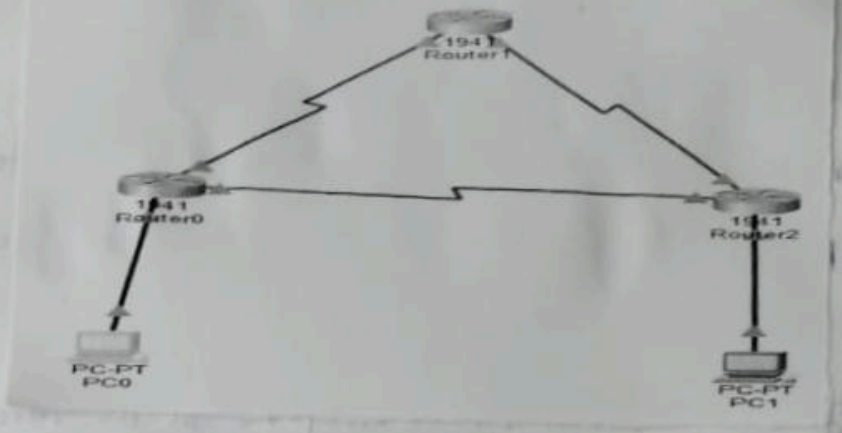
Ping statistics for 20.0.0.2:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 55ms, Maximum = 60ms, Average = 56ms

C:\>tracert 20.0.0.2

Tracing route to 20.0.0.2 over a maximum of 30 hops:

  1    0 ms      0 ms      0 ms      10.0.0.1
  2    11 ms     5 ms      5 ms      192.168.1.250
  3    25 ms     26 ms     25 ms     192.168.1.245
  4    22 ms     25 ms     20 ms     20.0.0.2

Trace complete.
```



Result:

RIP successfully configured and dynamic routing between PC0 & PC1 was verified.

3/10/24

Ex:no:14
Date: 23/10/22

Internetworking with routers in CISCO Packet Tracer Simulator.

a) Design and configure a simple Internetworking using router

Aim:-

To design and configure a simple internetwork using a router in cisco packet Tracer and test connectivity between two PCs.

Procedure:

Step 1: Configuring Router1:

* Select the router & open [L]
* Press enter to start configuring router1
* Type enable to activate the previladged mode.

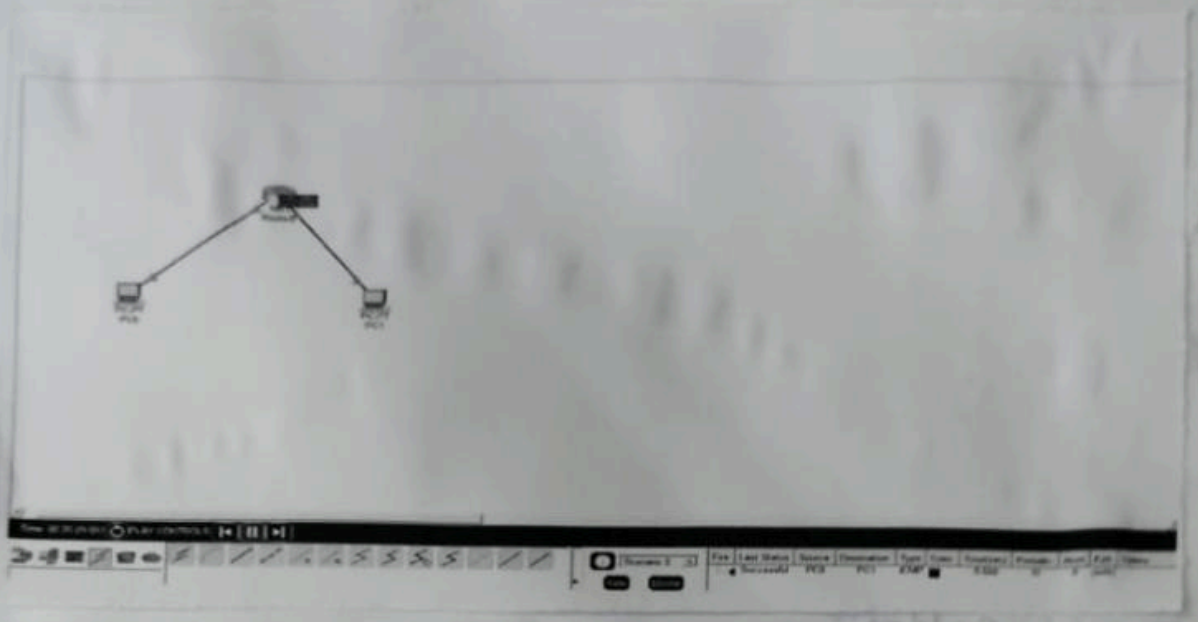
Step 2:

* Assign IP address to every PC in the network
* Select the PC, GO to the desktop and select IP Configuration & assign an IP address, default gateway subnet mask.
* Assign the default gateway of PC0 as 192.168.10.1
* Assign the default gateway of PC1 as 192.168.20.1

Step 3:

* Connect fast Ethernet 0 port of PC0 with fast Ethernet 0/0 port of router1 using a copper Straight-through cable.

* Connect fast ethernet 0 port PC1 with fast ethernet 0/1, port of router using a copper straigh through cable



Result:

The simple internetwork was successfully designed & configured using a router. Connectivity between PC0 and PC1 was successfully verified using PDU

13/10/21

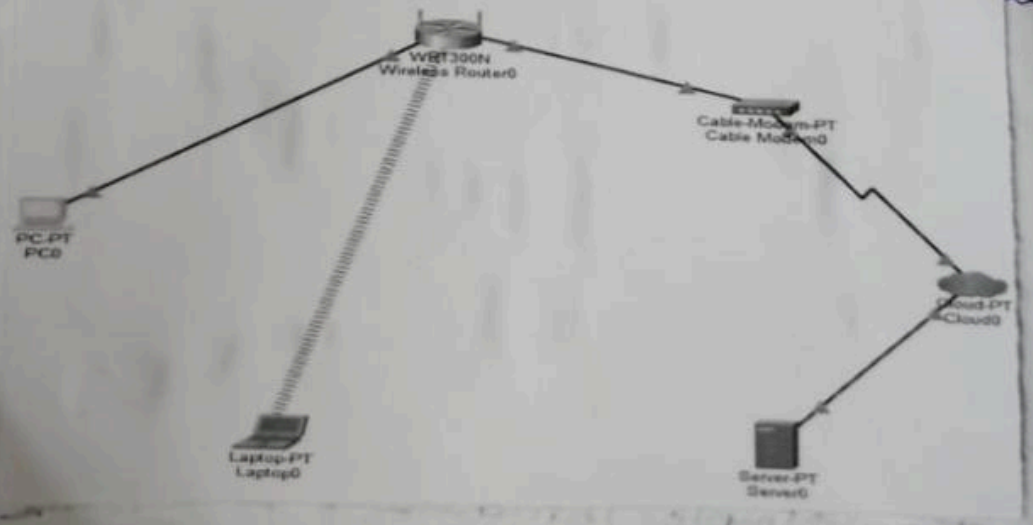
b) Design and configure an internetwork in your lab using switch, router & Ethernet cables.

Aim :-

To design and configure an internetwork using a switch, router and ethernet cables and verifies connectivity between computers

Procedure :

* open CISCO packet tracer and place computers, a switch and router in the workspace
* Connect the computers to the switch using ethernet cables.
* Connect the switch to the router using an Ethernet cable
* use the ping command to test connectivity between computers.
* c



Result :-

Thus an internetwork was successfully designed and configured using a switch, router and ethernet cables in Cisco packet tracer

Ex no: 15
Date: 13/01/25 Analyze various types of servers using websites

Aim:-

To analyze web server log file using the webalizer tool and study server traffic statistics.

Procedure:-

* Install and configure the webalizer tool on a system.
* obtain the web server log file, such as an Apache access log.
* Configure webalizer to read the server logfile.
* Execute the webalizer command to generate statistical reports
* open the generated HTML report in a web browser.
* Analyze the number of visitors, requests, page views, hits and bandwidth usage.
* Compare server activity using the generated reports.

Result :-

Thus, web server log files were analyzed succesfully using the webalizer tool, and server traffic statistics were studied.

13/10